REPORT · 2026年8月

中小型文物三维建模自动化系统 v1.1.8-r12 发行说明

H3D 项目组

2026 年 10 月 26 日
2026年8月

目 录

一、发行概览 4

二、本版改动总表 5

三、中误差口径（r12-1 ~ r12-5） 6

3.1 现象：报出的中误差比模型还大 . . . 6

3.2 根因：*1000 的语义前提从未成立 . . . 6

3.3 修复：三口径并行落盘 . . . 6

3.4 判据收紧：两条前提缺一不判 . . . 6

3.5 实现片段 . . . 7

3.6 报表、HTML 与论文表格的同步改写 . . . 7

四、结果条替代模态弹窗（r12-6） 9

4.1 为什么模态弹窗是错的 . . . 9

4.2 结果条的设计 . . . 9

五、强制重算入口（r12-7 ~ r12-8） 10

5.1 缺口：没有「明知产物在、但就是要重算」的表达 . . . 10

5.2 三个入口，同一份语义 . . . 10

5.3 边界：强制重算不动缓存 . . . 10

六、复用索引提升到用户级（r12-9 ~ r12-12） 11

6.1 原设计与它的天花板 . . . 11

6.2 新的位置与合并规则 . . . 11

6.3 跨项目复用是能力，也是误用风险 . . . 11

6.4 并发与持久化 . . . 11

七、验证与回归 13

7.1 测试 . . . 13

7.2 测试暴露出的三个坑 . . . 13

7.3 真机对照（口径部分） . . . 13

八、已知限制与下一步 14

一、发行概览

本次发行版本 **v1.1.8（修订 r12）**，打包日期 2026-10-26。上一版 v1.1.7 把「一次正确的运行被报成失败」这类**界面壳层**问题收口之后，还剩四项遗留，本版一次处理完。其中第一项不属于界面，属于**数据口径**——它决定了论文里能不能直接引用这个系统报的精度数字。

四项的关系可以用一句话概括：**前三项让「人」能正确操作，第四项让「机器」不再重复劳动。**① 中误差（M5）在纯照片场景下被乘了 1000 当毫米报，数值比模型本身还大 6 倍；② 每次跑完都弹模态窗打断操作；③ 产物明明在、就是想重算，界面一路跳过，只能手删目录；④ S1 的复用索引写在项目根里，同一批照片换个项目又要从头解一遍。

> **一句话结论**：本版首次让精度指标**可以跨件、跨批、跨尺度比较**——无激光锚点时不再报假毫米，改报「相对中误差（%对角线）」，无量纲、与尺度无关。同一件的 41.16「mm」 实为 **0.0412 模型单位**，即包围盒对角线 6.062 的 **0.68%**。

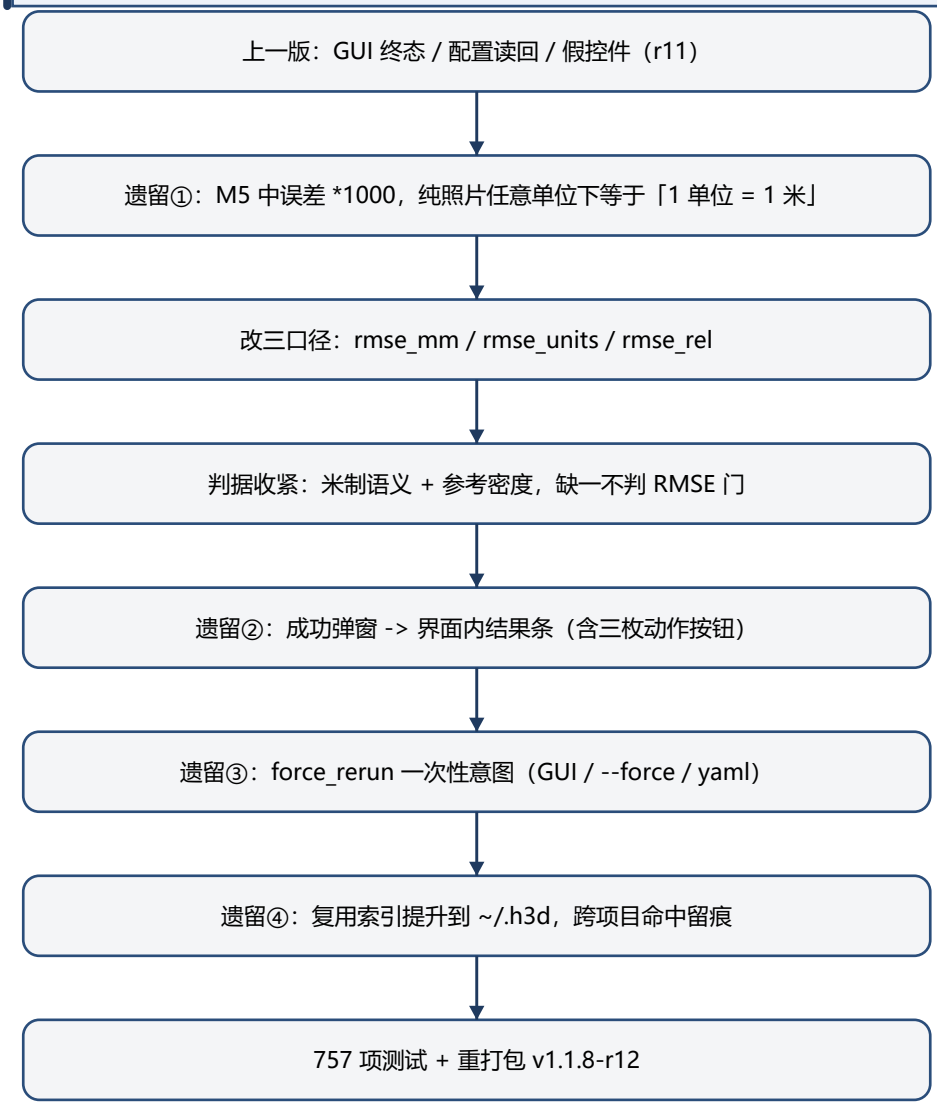


图 1 本版四项改动的依赖顺序：先定口径（否则后面的报表都在写错数），再改交互，最后动缓存范围

二、本版改动总表

编号	改动	关键数字 / 影响面
r12-1	M5 拆出 qc_rmse_units() 与 bbox_diag()	qc_rmse() 保留为兼容包装
r12-2	quality.run() 新增 metric_units 入参	返回 rmse_mm / rmse_units / rmse_rel 三口径
r12-3	无锚点时 rmse_mm 写 N/A	报表 / HTML / 论文表格同步改写
r12-4	check_quality() 无米制语义只判覆盖率	避免假毫米值把合格件判成不合格
r12-5	新增 quality.rmse_rel_warn（默认 0.02）	相对中误差超阈值走 WARNING，不参与硬判定
r12-6	结果条替代成功弹窗	文案三态 + 打开目录 / 打开报告 / 强制重算
r12-7	新增 runner.force_rerun 一次性意图	GUI 勾选（二次确认）/ --force / yaml，跑完自动复位
r12-8	_force_dense_restart() 用 deepcopy 副本	标志不泄漏到下一件与下一次运行
r12-9	复用索引提升到 ~/.h3d/s1_reuse_index.json	H3D_HOME 可覆盖；条目带 root / ts
r12-10	跨项目命中记 from_root / cross_project	并打 WARNING 提示确认是否同一件器物
r12-11	写前重读合并 + fsync + 原子替换	workers>1 时不互相覆盖；上限 500 条按 ts 淘汰
r12-12	项目级索引保留为兼容读 + 镜像写	老版本 exe 仍能读到自己那一份

三、中误差口径（r12-1 ~ r12-5）

3.1 现象：报出的中误差比模型还大

真机项目（纯照片场景，无激光扫描件）的 M5 报表写着：**中误差 = 41.16 mm。**而同一份 metrics 里，模型的包围盒对角线是 **6.062。**一个「误差」比被测量的对象本身大 6.8 倍，这在几何上不可能——除非单位搞错了。

指标	旧报表值	真实含义
reg_error	41.16（标注为 mm）	0.0412 模型单位，被误乘 1000
包围盒对角线	6.062	6.062 模型单位（尺度任意）
误差 / 对角线	—（未计算）	0.68 %
结论	「精度差」	实际 0.68%，属正常范围

3.2 根因：*1000 的语义前提从未成立

旧实现里 `qc_rmse()` 的末行是 `return v * 1000.0`，注释写着「米 → 毫米」。这个换算的**语义前提**是「模型的 1 个单位就是 1 米」——在有激光扫描件、由扫描仪给出真实尺度的场景 ① 下，这个前提成立；而在纯照片场景 ② 下，SfM 恢复的坐标是**任意单位**（尺度不可观测，没有度量基准），乘 1000 等价于**单方面宣称「1 单位 = 1 米」。**

> **这不是显示错误，是量纲错误。**它不会抛异常、不会告警、不会让覆盖率变差，只会安静地在报表里写下一个凭空的数字——而论文引用精度时，用的正是这个数字。项目里此前「禁静默降级」的规矩之所以必要，这条是又一例证。

3.3 修复：三口径并行落盘

不再在两种语义之间**猜**一种，而是三个口径**全部**写出来，让读者自己按场景选。三者的定义、单位与适用条件如下表。

键名	定义	单位	何时有值	用途
rmse_mm	表面点到参考点云的中误差	毫米	仅有激光锚点时	场景 ① 的绝对精度，可与规范值比对
rmse_units	同上，不做任何尺度换算	模型单位	永远有	原始量，任何二次计算都用它
rmse_rel	rmse_units / bbox_diag	无量纲（%）	有参考且对角线 > 0	唯一可跨件 / 跨批 / 跨尺度比较的口径
metric_units	本件是否具备米制语义	布尔	永远有	决定上面哪个口径是「可用」的
model_bbox_diag	模型包围盒对角线	模型单位	永远有	rmse_rel 的分母，供复核

`metric_units` 的判据是 `bool(task.raw_paths)`——「本件是否附带了扫描件」。这与双场景判定（S1 失败在场景 ① 只降级、在场景 ② 即整件失败）用的是**同一个谓词**，避免出现「一处认为是场景 ②、另一处认为是场景 ①」的分裂。

3.4 判据收紧：两条前提缺一不判

只改数值不改判据，会带来新的误判：一个 0.0412 的任意单位误差，若仍按「41.16 mm < 阈值」去比，几乎必然 fail。因此 RMSE 合格门加了两条前提，**必须同时成立**才参与判定，缺一则只判覆盖率。

前提	判定式	不满足时的行为
① 米制语义	metric_units == True	跳过 RMSE 门，只判覆盖率
② 参考足够密	参考点间距(mm) <= rmse_threshold_mm	跳过 RMSE 门（参考太疏时 RMSE 无意义），只判覆盖率

第二条是这次一并补上的：rmse_reliable 以前只是算出来记一笔，没有真正参与门控。实测在一个参考点间距约 50「mm」的夹具上，5 mm 阈值下rmse_reliable=False，此时若仍判 RMSE 门，结果只反映参考的稀疏程度，与被测模型无关。

3.5 实现片段

```
def qc_rmse_units(mesh, ref_pcd, samples: int = 50000) -> float:
    """表面中误差（**模型自身单位**）。

    .. warning:: 返回值没有任何尺度语义。``*1000`` 只有在「模型的 1 个单位
      就是 1 米」时才等于毫米，而这个前提只在有激光锚点的场景成立。
    """
    ...
    return float(np.sqrt(np.mean(np.square(dists))))


def bbox_diag(geometry) -> float:
    """包围盒对角线长度（模型单位），用作相对中误差的分母。"""
    ...


def qc_rmse(mesh, ref_pcd, samples: int = 50000) -> float:
    """旧接口，保留兼容：返回「毫米」，仅在米制语义下有意义。"""
    v = qc_rmse_units(mesh, ref_pcd, samples)
    return float("nan") if v != v else float(v * 1000.0)

rmse_units = qc_rmse_units(mesh, ref_pcd, cfg.samples)
diag = bbox_diag(mesh)
rmse_reliable = bool(metric_units) and (mps * 1000.0 <= cfg.rmse_threshold_mm)
rmse_rel = (rmse_units / diag) if (has_rmse and diag > 0) else float("nan")

metrics = {
    ...
    "rmse_mm":      rmse_units * 1000.0,
    "rmse_units":   rmse_units,
    "rmse_rel":     rmse_rel,
    "rmse_unit":    "mm" if metric_units else "模型单位",
    "metric_units": bool(metric_units),
    "model_bbox_diag": round(diag, 6),
}
```

3.6 报表、HTML 与论文表格的同步改写

口径变了而下游不改，等于白改。本次同步动了四处，全部指向同一个语义：**无锚点时请引用相对中误差。**

位置	旧	新
accuracy.csv 表头	仅 reg_error（mm）	插入 reg_error_rel「相对中误差(%对角线)」
reg_error 列值	始终为 rmse*1000 数字	metric_units=False 时写 N/A

位置	旧	新
report.html	表头 11 列，脚注统一说「mm」	表头 12 列；混排时行内追加「(模型单位)」；脚注分场景说明
thesis_experiment_table.csv	「M5中误差(mm)」一列	保留原列（无锚点写 N/A）+ 新增「M5相对中误差(%对角线)」

混排的处理值得单说：同一次批处理里可能既有场景 ① 件又有场景 ② 件。此时「中误差」一列里会同时出现 0.42（mm）与 41.16（模型单位），两个数放在一起看会误导。因此 HTML 表在检测到同表混排时，会在非米制行末尾追加「(模型单位)」，让人一眼看出这两列不是同一个东西。

四、结果条替代模态弹窗（r12-6）

4.1 为什么模态弹窗是错的

模态弹窗的问题不在于「弹」，而在于**它把成功与失败当成同一种事件。**失败必须被打断（不给确认就不该继续），但成功的处理方式恰恰相反：用户想立刻去看产物。旧实现两者共用 QMessageBox，于是每次成功都要先点一次「确定」，再自己去资源管理器翻输出目录。

4.2 结果条的设计

结果条是阶段灯下方的一条常驻控件（QFrame#resultBar），运行前隐藏、结束后显示，**不遮日志、不夺焦点。**文案三态与按钮启用规则：

终态	结果条文案	按钮启用
全部完成	全部 N 件完成，用时 …	打开输出目录 / 打开报告 / 强制重算
全部跳过	无需重跑：全部 N 件已是最新（…）	同上；并追加「如需重算：勾选左侧「强制重算」后再点一次」
部分跳过	完成 N 件，其中 M 件已是最新	同上
有失败	N 件失败，详见日志（弹模态确认）	打开输出目录

「打开报告」按钮的启用是**按产物存在性**判定的：没有 report.html 就不启用，避免点了之后打开一个空文件夹或弹系统错误——按钮能点却做不了事，比按钮是灰的更容易让人怀疑程序坏了。

失败路径**仍然弹模态。**这是唯一值得打断的场景：出错必须被看见，而不能指望用户主动去看一条可能已经滚过去的日志。

五、强制重算入口（r12-7 ~ r12-8）

5.1 缺口：没有「明知产物在、但就是要重算」的表达

断点续跑在正常流程里是对的，但它无法表达一种很常见的意图：**我改了参数，我知道产物在，我要重跑。**此前的唯一办法是手工删 `30_dense/` 或 `80_export/`，删多了会连带丢掉不想重算的东西，删少了又不生效。

5.2 三个入口，同一份语义

入口	写法	适用
界面	左侧勾选「强制重算（忽略断点续跑）」（二次确认）	交互使用
命令行	`H3D.exe --batch 配置.yaml --force`	脚本 / 批处理
配置	`runner.force_rerun: true`	固化到项目

关键在**「一次性」**：跑完自动复位为 `false` 并把 `config.yaml` 写回，下一次运行回到正常断点续跑。若不复位，用户勾过一次之后会永远在重算，而且找不到原因——这类「状态泄漏」在本项目已经踩过一次（`force_redense` 就地改共享配置），所以这次两处一起防：

```
def _force_dense_restart(cfg):
    """返回一份把 ``sfm_solve.force_redense`` 打开的**副本**。

    不能就地改：``self.cfg`` 是整批共享的对象，就地改会把标志泄漏给
    下一件任务与下一次运行。
    """
    import copy
    try:
        out = copy.deepcopy(cfg)
        out.sfm_solve.force_redense = True
        return out
    except Exception as exc:
        log.warning(...)
        return cfg
```

5.3 边界：强制重算不动缓存

这是最容易被误解的一点，因此日志里显式写清。`force_rerun` 只作用于**两处**：manifest 的 `is_done()` 跳过判定，以及 `sfm_solve.force_redense`（强制重做稠密）。而 S1 的照片指纹复用与 M4 的图集缓存是**参数键控**的——参数没变就仍然命中。

> **为什么不连缓存一起清？**因为那样每次点「强制重算」都要白等半小时的 SfM。真正的需求是「重跑流程」，不是「销毁证据」。若确实要重解 SfM，应显式删除 `99_runs/s1_reuse_index.json` 里对应条目，这是**两个不同量级**的操作，不该合并成一个按钮。

启动时的日志会明说这件事，避免用户以为「点了强制重算，结果还是复用了，是不是没生效」：

```
[!] 强制重算已开启（runner.force_rerun=true）：忽略断点续跑，N 件的全部
    阶段重跑。注意：M4 图集缓存与 S1 照片指纹复用仍按参数判定——
    参数未变则照常命中（否则每次都要从头解 SfM）。
```

六、复用索引提升到用户级（r12-9 ~ r12-12）

6.1 原设计与它的天花板

v1.1.5 引入的照片指纹复用，把「5 件同照片集各解一次 SfM」从 **1801 s × 5** 压到 **1801 s + 0.2 s × 4**。但索引写在 <项目根>/99_runs/s1_reuse_index.json，天生只看得到本项目：同一批照片在另一个项目目录里重新建一次，还是会从头解。而实际工作中「同一批照片开多个项目」很常见——换个流派试一版参数、拆成单件复核、给别人复现——每次都要重付 30 分钟。

6.2 新的位置与合并规则

层	路径	角色
用户级（主）	~/.h3d/s1_reuse_index.json	读写的权威副本；跨项目可见。可用 H3D_HOME 覆盖
项目级（辅）	<项目根>/99_runs/s1_reuse_index.json	兼容读取 + 镜像写回，供旧版本 exe 使用

用 H3D_HOME 而不是 %LOCALAPPDATA%：后者是 Windows 专有路径，而本系统在 Linux 上也要能跑（CI 就在 Linux 上跑测试）。H3D_HOME 还给三件事提供了入口：便携版把索引放 U 盘、测试隔离（不让测试写进真实家目录）、多用户互不干扰。

合并顺序与冲突规则**必须写死**，否则同一指纹在两处给出不同结果时会随机：同指纹以用户级为准；老条目没有 root 字段时，认定它属于其所在项目的根（这是历史条目的唯一合理解释）。

6.3 跨项目复用是能力，也是误用风险

跨项目命中能省半小时，但也可能命中**另一个器物**的照片——如果两件器物的照片恰好逐张相同（例如同一批素材被误放进两个项目），复用会把 A 的几何装进 B 的目录，而且**结果看起来完全正常**。因此命中时必须留痕：

```
"s1_reuse": {
    "hit": true,
    "from_root": "E:/兵马俑照片/最终测试",
    "cross_project": true,
    ...
}

WARNING  S1 复用命中来自**另一个项目**：E:/兵马俑照片/最终测试
         交付前请确认这是同一件器物的重复拍摄，而不是另一件器物的同批素材。
```

三处留痕（日志 / manifest 的 s1_reuse / 报表），与 v1.1.5 设的规矩一致：**任何「省了事」的判定都必须在三处可见**，否则「结果从哪来」就无法复核。

6.4 并发与持久化

风险	处理
多 worker 同时写索引互相覆盖	写前重新读一次再合并，然后落盘
写到一半被杀进程 -> 文件截断	tmp + fsync + os.replace 原子替换
索引无界增长	上限 500 条，按 ts 淘汰最旧
文件损坏导致整批跑不起来	改名 .corrupt-<时间戳> + WARNING + 空索引重跑

这四条不是预设的谨慎，而是本项目已经各自踩过一次：manifest 被 taskkill 截断过（导致项目永久跑不起来）、缓存文件损坏过、同一根两批并发互毁过。规则是**从故障里长出来的**，不是清单式补的。

七、验证与回归

7.1 测试

全量 **757 项通过**（较 v1.1.7 的 717 项 **+40**），唯一 warning 是 py4dgeo 的 `OMP_NUM_THREADS` 提示（既存，与本版无关）。

测试文件	项数	覆盖
test_quality/test_rmse_units.py	14	三口径一致性 / 无锚点写 N/A / 判据不得因假毫米 fail / 报表四处口径
test_runner/test_force_rerun.py	6	一次性语义 / deepcopy 不泄漏 / 跳过判定被绕过
test_runner/test_sfm_reuse_user_index.py	13	用户级位置 / 跨项目命中留痕 / 老条目迁移 / 500 条上限
test_gui_result_bar.py	10	结果条可见性 / 三态文案 / 按钮按产物启用 / 强制重算开关

7.2 测试暴露出的三个坑

写测试的过程本身修了三处，值得记下来，因为它们都是**测试写法**而非产品缺陷——但每一条都差点让测试变成「永远为真的假断言」。

坑	表现	原因与改法
`isVisible()` 恒 False	结果条明明已显示，断言却过不了	父窗口未 `show()` 时子控件 `isVisible()` 永远 False；改用 `isHidden()`
两次 `qc_rmse` 调用不等	54.189 vs 54.249，差 0.1%	`sample_points_uniformly` 每次随机播种；断言加 1% 容差
「更密的参考应更准」方向搞反	以为 RMS 会略小于平面间距，实测 0.0542 > 0.05	横向偏心占主导；断言改为区间 `0.05 ~ 0.06`

> **一条可复用的教训**：测试的价值在于**能失败**。上面第三条如果当时顺手把断言放宽成 < 0.1，这条测试就永远不会报警——而它想守的恰恰是「几何量级不能错一个数量级」。断言应当锁**行为本质**，不是锁「不报错」。

7.3 真机对照（口径部分）

同一件纯照片件，修改前后对同一份中间产物的引用方式：

量	修改前	修改后	说明
报表「中误差」	41.16 mm	N/A（无米制语义）	不再输出凭空的毫米
rmse_units	—（未输出）	0.0412	原始量，可复算
rmse_rel	—（未计算）	0.68 %	口径可引用
包围盒对角线	6.062（未进报表）	6.062（进报表）	分母可复核

八、已知限制与下一步

1) **场景 ① 的绝对精度仍未在真机标定过**：本版只把口径分清，有激光锚点时的 rmse_mm 数值是否正确，取决于扫描件与照片重建的配准精度，需要在有真值（如标定球 / 三坐标）的样品上验一次。

2) **结果条的「打开报告」依赖 report.html 存在**：若用户只跑了单件、没走报表阶段，按钮为灰。这是有意的（避免打开空目录），但若后续增加「单件快速预览」功能，应改为按可用的最高优先级产物判断。

3) **强制重算仍无法「只重算某一件」**：当前是整批。按件强制重算需要把 force_rerun 从布尔扩成集合，并让 GUI 在结果条上按行勾选——工作量不大，但会动到 item_id 与配置序列化的接口，暂缓。

4) **跨项目复用的误用防护只有 WARNING**：更强的做法是把器物编号写进照片指纹（来自 00_raw 的目录名），跨项目命中且编号不同时直接拒用。这需要先确定编号来源是否可信——若目录名本身就是随手起的，拒用反而会挡住合法复用。

5) **真机待验**：本版在真实多件批处理上的端到端表现（尤其跨项目复用命中率）尚需实测一次；另 GUI 弹窗点选与多进程抢锁两项仍待真机确认。

> **本版可复用的一条工程经验**：**「单位」是接口的一部分，不是注释里的一句话。**`*1000` 写在一个函数末尾，看起来只是换算，实际上它把「模型 1 单位 = 1 米」这个假设编码进了返回值——而这个假设只在一种场景下成立。凡是「同一个量在不同场景下语义不同」的地方，正确做法不是猜一种，而是**三种口径一起给出来，并把「哪个可用」也变成一个字段。**